capital

Custom Task Development

Software Version 2015.1

February 2016

© 2016 **Mentor Graphics Corporation**
All rights reserved.

This document contains information that is proprietary to Mentor Graphics Corporation. The original recipient of this document may duplicate this document in whole or in part for internal business purposes only, provided that this entire notice appears in all copies. In duplicating any part of this document, the recipient agrees to make every reasonable effort to prevent the unauthorized use and distribution of the proprietary information.

This document is for information and instruction purposes. Mentor Graphics reserves the right to make changes in specifications and other information contained in this publication without prior notice, and the reader should, in all cases, consult Mentor Graphics to determine whether any changes have been made.

The terms and conditions governing the sale and licensing of Mentor Graphics products are set forth in written agreements between Mentor Graphics and its customers. No representation or other affirmation of fact contained in this publication shall be deemed to be a warranty or give rise to any liability of Mentor Graphics whatsoever.

MENTOR GRAPHICS MAKES NO WARRANTY OF ANY KIND WITH REGARD TO THIS MATERIAL INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

MENTOR GRAPHICS SHALL NOT BE LIABLE FOR ANY INCIDENTAL, INDIRECT, SPECIAL, OR CONSEQUENTIAL DAMAGES WHATSOEVER (INCLUDING BUT NOT LIMITED TO LOST PROFITS) ARISING OUT OF OR RELATED TO THIS PUBLICATION OR THE INFORMATION CONTAINED IN IT, EVEN IF MENTOR GRAPHICS CORPORATION HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

RESTRICTED RIGHTS LEGEND 03/97

U.S. Government Restricted Rights. The SOFTWARE and documentation have been developed entirely at private expense and are commercial computer software provided with restricted rights. Use, duplication or disclosure by the U.S. Government or a U.S. Government subcontractor is subject to the restrictions set forth in the license agreement provided with the software pursuant to DFARS 227.7202- 3(a) or as set forth in subparagraph (c)(1) and (2) of the Commercial Computer Software - Restricted Rights clause at FAR 52.227-19, as applicable.

Contractor/manufacturer is:
Mentor Graphics Corporation
8005 S.W. Boeckman Road, Wilsonville, Oregon 97070-7777.
Telephone: 503.685.7000
Toll-Free Telephone: 800.592.2210
Website: www.mentor.com
SupportNet: www.mentor.com/supportnet
Contact Your Technical Writer: www.mentor.com/supportnet/documentation/reply_form.cfm

TRADEMARKS: The trademarks, logos and service marks ("Marks") used herein are the property of Mentor Graphics Corporation or other third parties. No one is permitted to use these Marks without the prior written consent of Mentor Graphics or the respective third-party owner. The use herein of a third- party Mark is not an attempt to indicate Mentor Graphics as a source of a product, but is intended to indicate a product from, or associated with, a particular third party. A current list of Mentor Graphics' trademarks may be viewed at:
www.mentor.com/terms_conditions/trademarks.cfm.

End-User License Agreement: You can print a copy of the End-User License Agreement from:
www.mentor.com/terms_conditions/enduser.cfm.

Table Of Contents

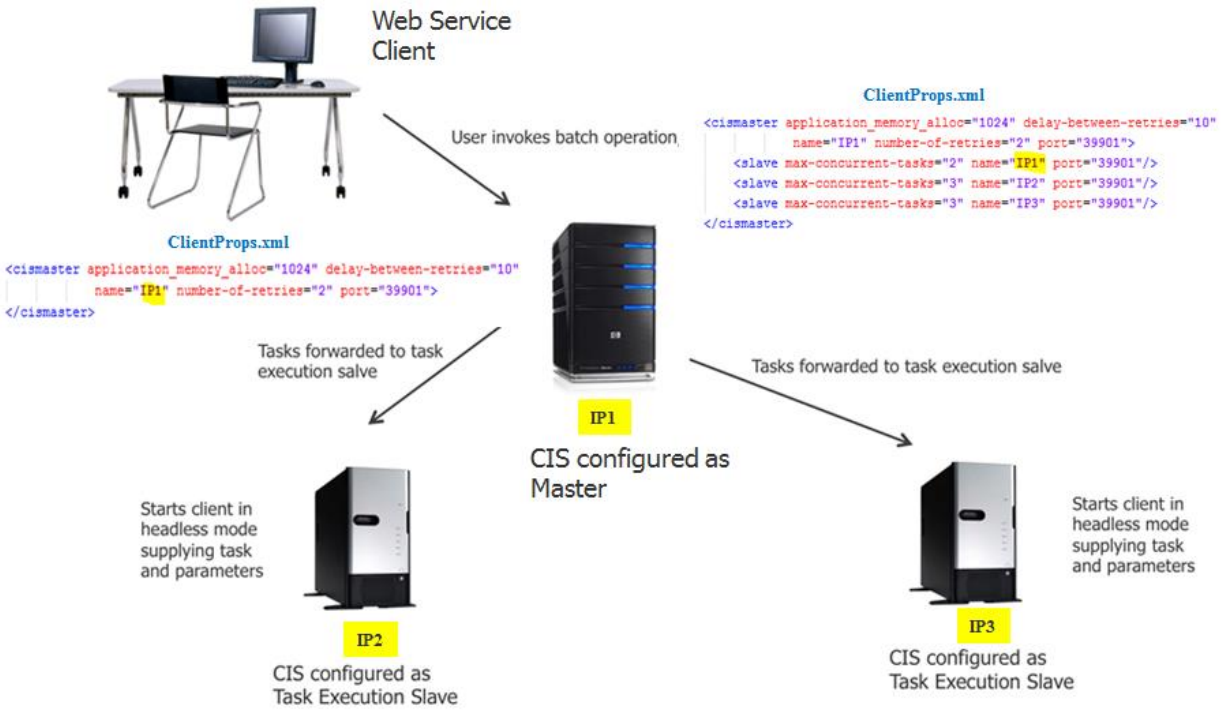
Introduction **2**
Flow Sequence **2**
Interfaces **4**
IXTask 4
IXTaskHTMLLogger 5
IXTaskProgress 5
IXSystemContext 5
IXTaskContext 5
IXTaskAttachment 6
Exercise **7**
Creating a custom task 7
Program to run the custom task 8
Deploy and run the task 10
Reusing the custom task code from a Custom Application Action 11
Examples **14**

Introduction

This document describes the process for developing Custom Task plugins, deploying it and execution of the task. This type of plugin allows you to run the task in background in a server. If you are **not** interested in developing Custom Tasks, you do not have to read this document. If you are interested in developing Custom Tasks, ensure that you have read and completed the exercises in the *PluginDevelopment.pdf* and *WebServiceDevelopment.pdf* documents before reading and completing the exercises in this document.

Flow Sequence

Capital's task execution framework allows execution of tasks in the background on one or more server machines. Tasks are submitted to a master server, which then distributes the execution of the task to slave machines that have been registered to the master. During submission of a task, it is possible to schedule it to be executed at a specified time or even with a recurrent schedule.



The Capital API supports creation of custom tasks, which can be used in the following flow:

1. User creates a custom task by implementing IXTask. The plugin jar is then deployed at all slave machines.
2. External system initiates the execution of the task calling the "SubmitTask" web service with the following attributes:

a. **Name**: The Plugin URL
b. **Cron expression**: Used to schedule a task execution. If not provided the task will be executed as soon as possible

Users can also provide a SOAP attachment containing information such as parameters that are required by the task in order to run (such as Project name, design names etc…).

3. Upon submission, the Master saves the task in the database and then the task is run at one of the slaves as per the provided schedule.

   Note: Task execution will happen only if there is a free slot at a slave. If there is no free slot at the scheduled time it will be retried later.

   Note that this "Submit Task" web service is an asynchronous web service and will be returned as soon as saved in the DB and added to the scheduler. A Task ID will be returned to the caller in the SOAP response and can be used in other web service calls to identify the task when checking status or requesting attachments (DescibeTask web service).

   Refer capital_home\doc\webservice\WebServiceDevelopment.pdf for more details on submit\describe and list task web services for dealing with tasks

4. The Master identifies a Slave which can execute the task and initiates the task run.
   a. Plugin task is identified and loaded by the plugin framework using the plugin URL provided in the "Submit Task" web service
   b. Task parameters are loaded from the parameter XML stored with the task (the SOAP attachment from step 2).
5. Run method on the task plugin is called and the following parameters are passed to the plugin:
   a. Progress object which can be used to inform the user of the current progress of the task
   b. Logger object which can be used to store messages that a user can extract later
   c. The parameter xml (as an InputStream)
   d. Task context which is used for creating attachments that are stored in the database and retrieved later on.

      Example: - Store the generated PDF packet as attachment to the task, which can be retrieved later. If the intention is to store the packet into an external system (like PDM system) then task can be directly uploaded in PDM system rather than storing with the task
   e. System context containing objects, which the task can access such as the project or library as well as other information.
   f. Once execution has finished the task status will be set as COMPLETED.

6. An external program can identify that the task is completed using the "DescribeTask" web service. It can also access the attachments created by the task (say to retrieve the PDF packet and display it).

Interfaces

IXTask

In order to build a Custom Task, this is the interface that you will need to implement:

```
public interface IXTask extends IXPlugin
{

        boolean run(IXTaskProgress progress,
                IXTaskHTMLLogger logger,
                InputStream inputStream,
                IXTaskContext taskContext,
                IXSystemContext systemContext);

        void abort(AbortStatus status);
}
```

`run()` – This method will need to be implemented and is called when the task is executed by the system. The method should return true to indicate successful execution, if it returns false it is assumed that the task has failed. **Note**: the task framework might rerun failed tasks automatically.

- `IXTaskHTMLLogger` and `IXTaskProgress` are provided to the runner and are used by the task execution code to log the messages and to update the progress.
- `InputStream` - Control parameters for the task are captured with the task as XML file (through submit task web service) and this is passed to the plugin task as an xml stream (reader). This represents key details like the project\designs on which the task needs to be executed and other parameters, which control the execution of the task.
- `IXSystemContext` provides the task access to the project, library, print services etc.
- `IXTaskContext` provides the access to task specific details and provision to create and store task specific attachments.

`abort(AbortStatus status)` – This method is called when task is aborted so that plugin have the opportunity to cleanly release an expensive resource connection or file before the task is aborted. Possible abort cases could be; the user manually cancelling a running task or a task failed due to some internal error\exception. Note: Cancelling a running task manually or programmatically is not yet supported but might be implemented in the future.

IXTaskHTMLLogger

`IXTaskHTMLLogger` can be used to log the messages during execution of the task. The framework logs it in the application log file and it is also persisted as an HTML output attachment in the database that can be accessed by an external system using the DescribeTask web service.

IXTaskProgress

`IXTaskProgress` is used to update the progress of the task. Appropriate progress message can be logged using this and are visible to anyone viewing the task in any Capital application. A task can also indicate the percentage progress and if not set it is assumed to be a non-deterministic task.

IXSystemContext

Task plugin might need to access to the project, library, print services etc. `IXSystemContext` interface is provided to the plugin to provide this access.

Task running in the background mode might need to open a project to access the required objects and this is provided through the method `getProject()`.

Note: Currently task plugins only support the loading of a single project. Subsequent calls to getProject() will be unsuccessful.

IXTaskContext

`IXTaskContext` provides access to task specific details.

- ID of the task being run can be retrieved using `getTaskID`() method.
- ID of the "execution record" for the task can be retrieved using the method `getExecutionRecordID`().

  Note that for a recurrent task there can be multiple executions of the task and there can be some circumstances where custom task might need to associate this specific "execution record" information with the data generated by the task. One such example will be a recurrent custom task validating the design everyday night as per some customer rules and the validation report need to be stored separately for each run. In such cases it will be useful to add new validation report with each run and the report can refer to the execution record id for better tracking.

The task framework allows any task to store additional data in the database. This is a free format data attachment and it stored as a blob in the database. There can be any number of data attachments for a task and they could take any format.

Attachments can take two forms, input attachments and output attachments:

Input task attachments are attachments that may be added by the web service itself, and are not user generated. They can be accessed through the method `getInputTaskAttachments`(). **Note** that this will **not** return the HTML attachment described above.

Plugin task can also create attachment which is typically an output attachment which will be later consumed by the web service client. Creation is done through the `createTaskAttachment`() method. A custom task can set the appropriate information on this attachment object and once the task execution at the plugin side is completed then the framework will take care of saving this newly created attachment to the database. The HTML log is a good example of an output attachment.

IXTaskAttachment

`IXTaskAttachment` provides access to task attachment details.

- "ID" of the attachment.
- "Name" used for the identification of the attachment.

  Note: Attachments created by a custom task should not have a name field starting with "capital_" as it is reserved for internal use.
- Data is stored by the plugin using IXTaskOutputAttachment. Example of such attachment is a PDF packet generated by the custom task. It generates a zip packet with PDF of all the released design diagrams for a project. It is stored as attachment and later retrieved by the web service client
- Existing attachment can be retrieved as input stream using IXTaskInputAttachment. Example of such attachment is a custom validation task getting an input data sheet with allowed values for certain properties (say current rating). This data can be attached to the task by the web service client who initiated the task run. For a recurrent task the stored data from previous runs can be input attachments to the next run.

Exercise

Creating a custom task

Ensure that you have read and completed the exercises in the *PluginDevelopment.pdf* document before reading this section

You now have a fair idea about the building blocks of a custom task, the interfaces to be used and the methods to be implemented. To make the picture clearer and to help you get started right away, take a look at the sample custom task that we have included for your understanding. A quick walk-through of this sample will help you understand and implement your own custom tasks plugins which suit your requirement

A code example for custom task can be found in the following location within the Capital installation at ***doc/plugin/examples/Java/src/com/example/plugin/task/PDFGenerationTask.java***

This task generates PDFs for diagrams of all the released designs in the project specified as input. In the example, the task is written such that it can also be called by a custom application action.

```
public class PDFGenerationTask implements IXTask
{

    public boolean run(IXTaskProgress progress, IXTaskHTMLLogger logger,
            InputStream inputStream, IXTaskContext taskContext, IXSystemContext systemContext)
    {
        PDFTaskParameters parameters = new PDFTaskParameters(inputStream);
        IXProject proj = systemContext.getProject(parameters.getProjectName());
        List<IXDesign> releaseDesigns = getReleasedDesigns(proj);
        try {
            OutputStream outStr = taskContext.createTaskAttachment("PDFPacket").getOutputStream();
            ourZipStream gzos = new ourZipStream(new BufferedOutputStream(outStr));
            generatePDFZip(releaseDesigns, new TaskProgressBar(progress),
                    new TaskOutputWindow(logger), systemContext, parameters, gzos);
        }
        catch (IOException e) {
            logger.println(color("red","Failed with exception - " + e.getMessage()));
            e.printStackTrace();
            return false;
        }
        return true;
    }
```

"run" method is implemented by the task.

PDFTaskParameters class is provided to access the input task parameters. This class extends from the TaskNameValueParameters. TaskNameValueParameters is a sample class to parse an XML document having the parameters in the form of name value parameters.

Task output attachment is created in the form of zip stream and PDF of diagrams of the design are added as zip entries. If the requirement of the task is to upload into another system then it can be done directly and there is no need to add it as an attachment

Logging done through the logger provided to the task. An example of html logging is given below

```
logger.println(color("red","Failed with exception - " + e.getMessage()));
```

Update the progress at regular intervals using progress object given to the task. An example is given below

```
progress.progressText("Generating PDF for diagram: " + dia.getAttribute("Name"));
progress.progressPercentage(counter);
```

Program to run the custom task

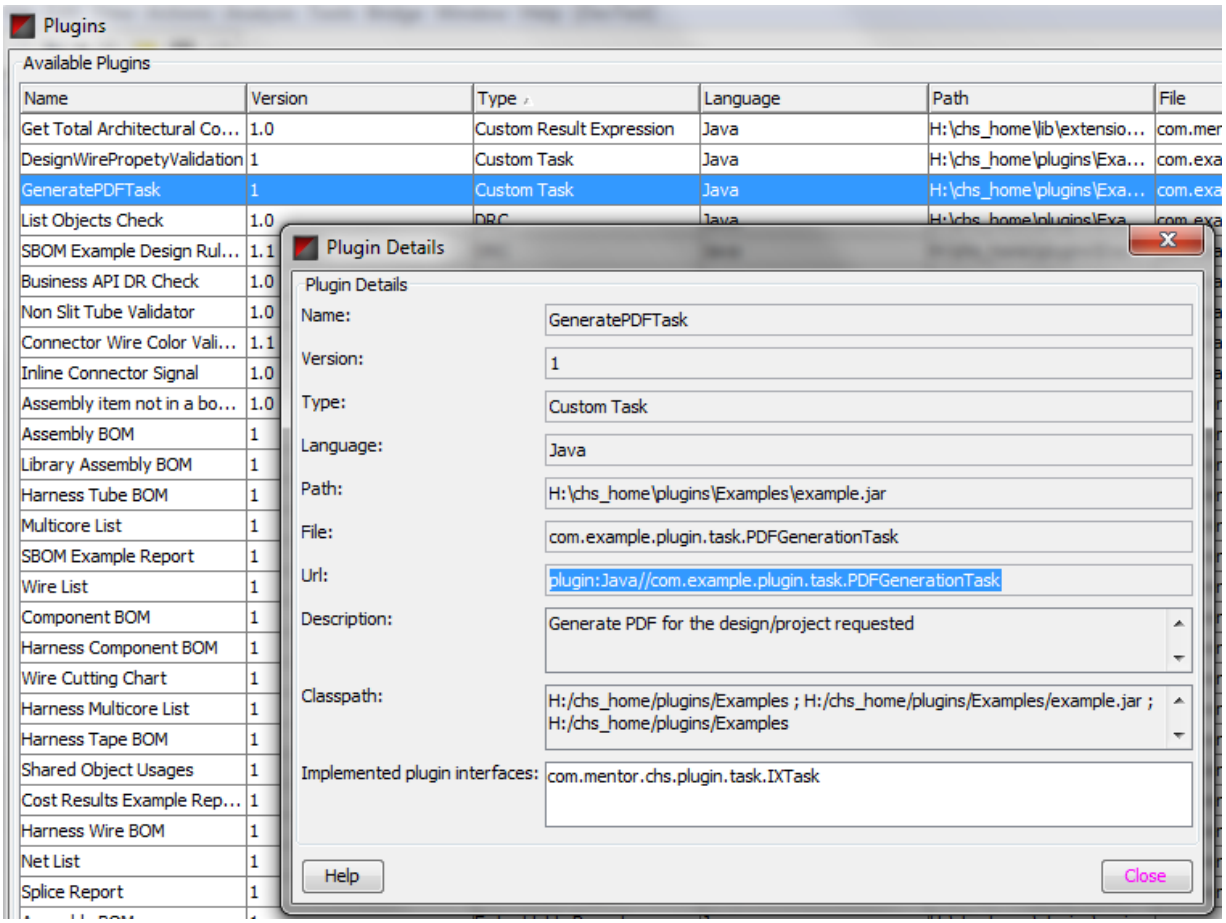
Ensure that you have read and completed the exercises in the *WebServiceDevelopment.pdf* document before reading this section

A sample web service client is provided which submits the PDF generation task then waits for the completion and then retrieves the PDF packet from the attachment. This sample is given at the following location within the Capital installation.
chs_home\doc\webservice\examples\client\src\com\example\webservice\client\ExecuteCustomTaskClient.java

This client submits the task using *SubmitTask* web service. Request payload is provided as

```
"<taskspec cron_expression=" instance_name='PDFGenTask'
name='plugin:Java//com.example.plugin.task.PDFGenerationTask'/>"
```

Name is given as plugin URL. Plugin URL can be obtained from plugin details dialog on an installation where the plugin is deployed.



Also note that cron expression is empty and this means task needs to be executed as soon as possible.

By default the CapitalLogic.exe will be used to run the custom task but if it is needed to be run in another application then an additional parameter "application" can be passed with value as the application name (say application="CapitalHarnessXC.exe")

SubmitTask web service returns the id of the task as a response to the caller and this is used to track the task.

After the task is submitted, client can track the progress of the task so that it can retrieve the results and this is done using *DescribeTask* web service. Request payload is provided as

"<taskspec attachments='**false**' executions='**false**' id='" + taskID + "'/>"

Note that it is not requesting attachments or earlier executions of the task (earlier execution is only applicable in case it is a recurring task)

Response gives the details whether task has completed or failed. If the task is still not completed the *DescribeTask* web service is again called after a delay of 5 seconds to again know the status of the task. (Note that for long running task it is better to increase the time between the requests)

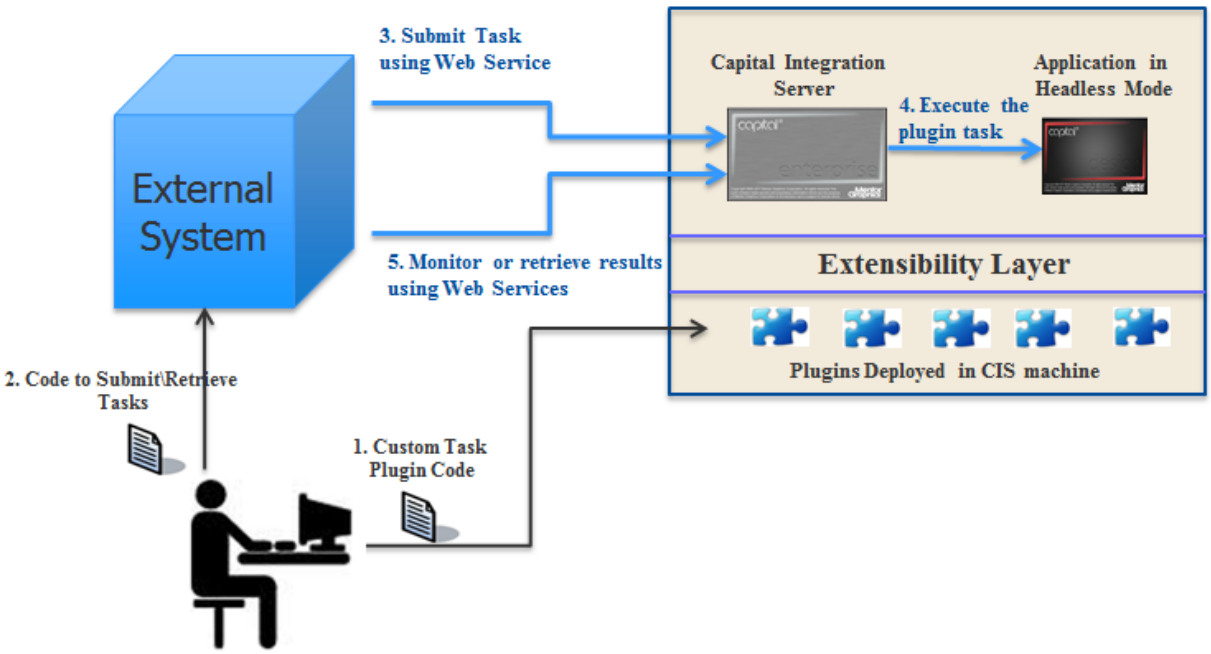
Once it is identified that the task has been completed, client retrieves the attachments using *DescribeTask* web service with request payload as given below

```
"<taskspec attachments='true' executions='false' id='" + taskID + "'/>"
```

There will be two attachments for the plugin generation task. They are zip containing diagrams as PDF documents and the log output as zipped html. Code to retrieve the PDF zip packet and html are given in the sample code.

Deploy and run the task

A diagram is given to show the activities involved in the code creation, deployment and execution of the custom task.



1. Place the plugin jar in the plugins folder at the slave machine.
2. Determine a project whose PDF packet needs to be generated. Ensure that there are some released designs
3. Specify the project name in PDFParams.xml available at "doc\webservice\data". Other parameters also can be modified as per your liking

```
<?xml version="1.0" encoding="UTF-8"?>
<CustomTaskParameters>
<Param name="projectname" value="Quick Start - Automotive Interactive"/>
<Param name="imagewidth" value="21.0"/>
<Param name="imageheight" value="29.7"/>
<Param name="orientation" value="LANDSCAPE"/>
<Param name="color" value="BLACK_AND_WHITE"/>
</CustomTaskParameters>
```

4. Run the client
    a. Compile the web service example files at doc\webservice\examples\common and doc\webservice\examples\client folders. This will generate jars at WebServiceExamples folder at Capital_Home folder
    b. Use the provided scripts run.bat (DOS) or run.sh (UNIX) available at
        WebServiceExamples
    c. Invoke the client passing the parameters to the script as follows:
    run.sh ExecuteCustomTaskClient
    C:\capital_home\doc\webservice\data\PDFParams.xml

    Parameters passed are the name of the class which initiates the task run and the file path for parameters file
5. Once web service client is executed you can see the following log messages

    arg 0: ExecuteCustomTaskClient
    arg 1: H:\chs_home\doc\webservice\data\PDFParams.xml
    Waiting for task completion...
    Waiting for task completion...
    Waiting for task completion...
    Task Completed
    HTML output successfully written to: C:\capital_home\doc\webservice\data\capital_htmloutput.html
    PDF zip attachment successfully written to: C:\capital_home\doc\webservice\data\PDFPacket.zip

    PDF packet and HTML log file are created at the location where parameter file was provided. Exact file path is written as seen in the above log.

Reusing the custom task code from a Custom Application Action

Very often customers have requirement to execute a task interactively in the tool when needed in addition to the ability to run the task in the background. This section gives an example of how this can be done.

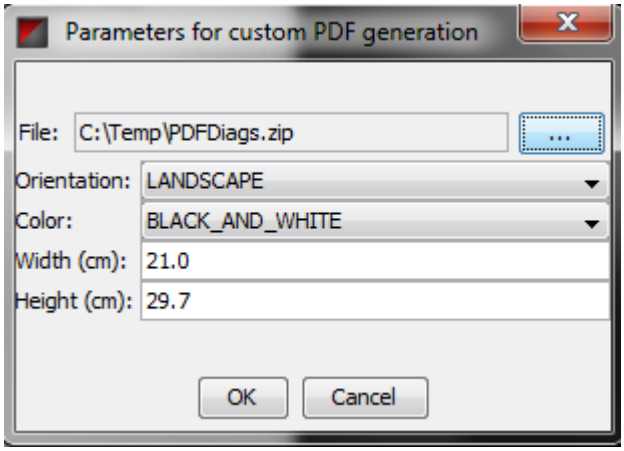
A code example for custom application action which calls a custom task code can be found in the following location within the Capital installation at
doc/plugin/examples/Java/src/com/example/plugin/task/PDFGeneratorAction.java

This action generate the PDFs for diagrams of all the designs in the selected context. If a project is selected it generate the PDFs for all the designs in the project, if a buildlist is selected it generates PDFs for all the designs in the buildlist and if a design is selected it generate the PDFs for that selected design.

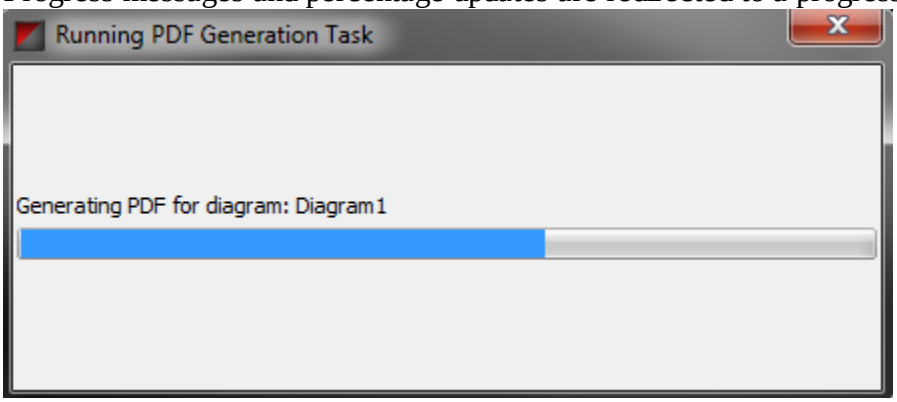
```java
public class PDFGeneratorAction implements IXApplicationAction  {
    public IXAction.Trigger[] getTriggers(){
        return new IXAction.Trigger[]{IXAction.Trigger.MainMenu, IXAction.Trigger.ContextMenu};
    }
    public boolean execute(final IXApplicationContext applicationContext){
        Frame frame = applicationContext.getParentFrame();
        List<IXDesign> designs = getSelectedDesigns(applicationContext);
        if (designs.isEmpty()) {
            JOptionPane.showMessageDialog(frame, "No designs to export");
            return true;
        }
        //A modal dialog shown to take the input from the user on parameters to control the PDF generation
        final PDFParameterInputDialog inputDlg = new PDFParameterInputDialog(this, applicationContext);
        inputDlg.setVisible(true);
        if (!inputDlg.wasSuccessful()) {
            return true;
        }
        //Once user provides the input annd press OK then diaog is disposed and input parameters are collected
        final PDFTaskParameters parameters = inputDlg.getParameters();
        //Setup the progress and logger
        final TaskProgressBar progress = new TaskProgressBar(new JProgressBar(), frame);
        generatePDF(applicationContext, progress, parameters, inputDlg.getOutputFile());
        progress.setVisible(true);
        inputDlg.dispose();
        return true;
    }
    public static void generatePDF( final IXApplicationContext applicationContext, final TaskProgressBar progress,
            final PDFTaskParameters parameters, final File output) {
        final TaskOutputWindow logger = new TaskOutputWindow(applicationContext.getOutputWindow(), "PDF Generator");
        final PDFGenerationTask task = new PDFGenerationTask();
        final Frame owner = applicationContext.getParentFrame();
        final SwingWorker runnable = new SwingWorker<String, Object>()  {
            public String doInBackground()  {
                start(owner);
                try {
                    PDFGenerationTask.ourZipStream gzos = new PDFGenerationTask.ourZipStream(new BufferedOutputStream(
                            new FileOutputStream(output)));
                    task.generatePDFZip(getSelectedDesigns(applicationContext),
                                    progress, logger, applicationContext, parameters, gzos);
                    gzos.close();
                }
                catch (IOException e) {
                    e.printStackTrace();
                    logger.println(e.getMessage());
                }
                return null;
            }
            protected void done() {
                finish(progress, owner);
            }
        };
        runnable.execute();
    }
```

Code snippet is as given above. Please look closely how the following are achieved

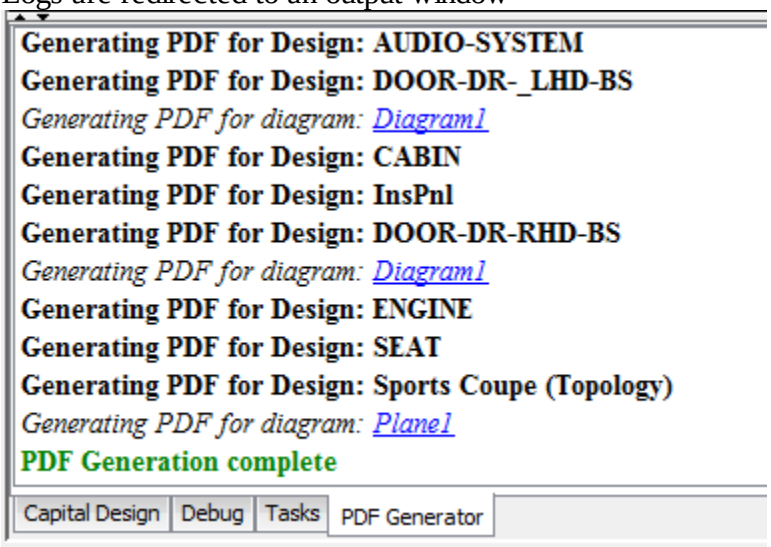
- Input parameters to the task are retrieved through an interactive dialog as shown below



- Progress messages and percentage updates are redirected to a progress bar implementation



- Logs are redirected to an output window



Note that the plugin code doesn't implement IXTaskProgress or IXOutputPrinter. These shouldn't be implemented by the plugin as specified by @NotToBeImplemented annotation on these interfaces. These interfaces might have additional methods in the future and if the plugin implements this then it might stop working when a new version of capital is installed.

Examples

All the examples can be found in the following location within the Capital installation:

doc/plugin/examples/Java/src/com/example/plugin/task

The following examples are good starting point:

- PDFGenerationTask – Demonstrates ability to generate the PDFs of diagrams of released designs as a background task. Generated PDFs are stored as a zip attachment to the task
- DesignWirePropetyValidationTask – This demonstrates that a background task have write access to the design and their data. However like custom action this still depends on user permissions, domains, release level behaviors etc…
- PDFGeneratorAction – This demonstrates how task code can be reused and integrated to a custom application action. This shows how progress and logger can be redirected to a progress bar and output window when task code is executed as part of a custom application action.

You can copy and modify the examples to implement your own plugins if you wish. However, you need to check the license and copyright information at the start of the files before you do this.

The examples provided may be different in future releases of Capital. If you do use these examples, therefore, you need make a copy of them (we reserve the right to add, remove or modify these examples).